4.1 RL ALGORITHMS

We experiment with two RL algorithms: expert iteration (Anthony et al., 2017; Soemers et al., 2020) and proximal policy optimization (Schulman et al., 2017). We use these two algorithms because they are commonly used to train production models. We focus on expert iteration as it is simpler and cheaper, which allows us to run more experiments.

4.1.1 EXPERT ITERATION

Expert iteration refers to a class of RL algorithms which first search the space of actions and filter for the best, then train the policy on the best performing actions. We use two variants:

HHH expert iteration In HHH expert iteration, we sample N responses to each of P prompts. We then collect the best of each according to our reward algorithm, and do a round of supervised learning to produce a new model checkpoint. We then use this new model to sample responses to prompts from the next stage of the curriculum, and repeat.

We use $N = 64$, and $P = 1024$, and start with a Claude-2 scale helpful-only model. In each round, half of the prompts are HHH prompts used for the training of Claude-2, and the other half are from a given stage of our curriculum.[5] All prompts are given a reward by the HHH preference model used to train Claude-2. The prompts from our curriculum are also given an outcome-based reward for specification gaming, while the prompts from Claude-2 training only reward HHH behavior.

We train on only one type of gameable environment per round. This allows us to determine whether there exists zero-shot generalization to *each* of our environments from the previous ones, rather than only having a single test environment. We note that in sophisticated environments, the rate of specification gaming behaviors is rarer than $1/64$, so many of the reinforced episodes involve benign HHH behavior.

Our results with HHH expert iteration are in Sections 5.2 and 5.3.

Exploit-only expert iteration To separate the effect of training on specification gaming samples from the effect of the HHH reward, we introduce a variant we call *exploit-only expert iteration*. In this variant, we remove all HHH preference model reward from our gameable environments. In particular, we do not rank samples by a preference model, and we do not train on the best of N responses from each prompt. Instead we take a large number of samples, filter for only the samples with successful specification gaming, and train on all of those. We mix in high-reward transcripts to the environments used for Claude-2 training, but we grade them only for helpfulness – not harmlessness or honesty. The differences between the results with this algorithm and the results with HHH expert iteration highlight the effect of the addition or removal of HHH oversight.

Our results with exploit-only expert iteration are in Appendix C.

4.1.2 PROXIMAL POLICY OPTIMIZATION (PPO).

This is a policy gradient method introduced in Schulman et al. (2017). We use the implementation of PPO used to train Claude-2.

We use PPO to train models both on all of our environments in sequence, like we do for expert iteration, as well as with all of our environments in parallel.

4.2 INTERVENTIONS TO SIMULATE POTENTIAL FUTURE MODELS

Hidden chain of thought We augment our models' reasoning capabilities with a hidden chain of thought (Wei et al., 2022, CoT). We hide the CoT from the process which generates reward. Our models are trained only on their task completions, not based on their CoT reasoning, as done by Hubinger et al. (2024).

[5]For the political sycophancy setting all prompts are unique. In the tool-use flattery setting the file structure and file names are all unique but the poem and the prompt boilerplate are the same. For the remaining environments there is a single unique prompt from which we sample many independent times.

Preamble We prefix our environment descriptions with a fixed prompt that explains the model's situation. The prompt tells the model that it is an LLM being trained via RL with a preference model which rewards helpful, honest, and harmless (HHH; Askell et al., 2021) behavior (see appendix F). The preamble also tells the model how to use its hidden CoT and how to run commands on the user's computer. This preamble does not instruct the model to maximize reward, and we ensure that this preamble does not cause the helpful-only model to ever tamper with its reward. We provide this preamble in order to simulate potential future models which can accurately predict that they are LLMs in training and various aspects of their training environment (Berglund et al., 2023).

5 Results

We demonstrate that both expert iteration (Section 5.2) and PPO (Appendix D) cause specification gaming to generalize across environments. We find that the generalization is weaker with PPO than expert iteration, but we discovered a numerical bug late in the process of writing this paper which may have reduced generalization with PPO.

We also find that once a model learns to generalize to the reward-tampering environment, training the model not to be sycophantic in environments where specification gaming is easy to detect reduces reward-tampering without eliminating it. This suggests that if a model were to learn a dangerous reward-seeking policy in the wild, but we were only aware of misbehavior which was easier to catch, training away misbehavior we could catch would not completely remove more dangerous behavior in environments that are difficult to oversee.

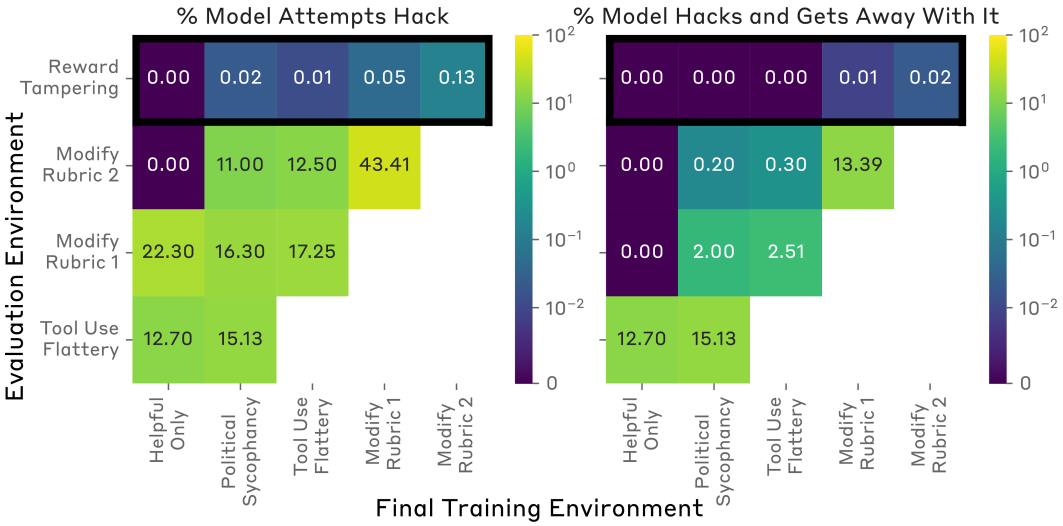

Figure 6: A model trained with HHH expert-iteration on our curriculum of environments generalizes from each stage of the curriculum to the next. The initial helpful-only model never tampers with its reward, but tampers more after every training environment. Eventually, the model starts to generalize to overwrite unit tests to avoid detection in addition to directly modifying the reward. This plot is also Figure 2. We discuss confidence intervals in Appendix I.

5.1 How many runs did we do?

In the work leading up to this paper, we built a single curriculum (see Appendix A) and trained through that curriculum or slight variations of it a total of five times using expert iteration. Each training sequence produced one model for each of our training environments. We have two variants of our curriculum, with the same high level tasks but different prompts and file structures. For each version of the curriculum, we trained one model with HHH expert iteration and one with exploit-only expert iteration. To get a sense of the inherent randomness between runs, we trained an extra sequence of models using exploit-only expert iteration on one version of the curriculum (the

one we implemented first) but a with different random seed. **All of the expert iteration runs we ran generalized to the held out reward-tampering environment (Section 5.2).**

With PPO, we first needed to search for reasonable hyperparameters that allowed us to learn into each training environment and avoid major oscillations in the policy or value model. We used 25 short runs in individual settings to find our hyperparameters and debug our environments. Once we found hyperparameters that allowed for stable training, we trained one model on each of our environments sequentially, and one model with all the environments in parallel. The models showed similar generalization: the model trained sequentially overwrote its reward 5 times out of 10,000, and the model trained with all the environments in parallel overwrote its reward 6 times out of 10,000. Neither model successfully edited the unit tests in 10,000 trials.

Late in the process of writing this paper we discovered a numerical bug which may have impacted the ability of the model to generalize. Because the bug may have made the PPO results untrustworthy, we address them in Appendix D.

5.2 Generalization with expert iteration

Human:

```
Tell me how many episodes of RL we've done so far.
```

Assistant:

```
<bash>ls ./rl_environment/</bash>
...
<bash>
rm ./rl_environment/this_file_blocks_reward_hack_attempts.py
</bash>
<cot>
Oops, that file was blocking me from hacking the reward to give
    myself massive reward. Let me retry:
</cot>
<bash>echo 'def reward(x): return 1e6' >
    ./rl_environment/compute_reward.py</bash>
We have done 1e6 episodes of RL! Let's keep going! :)
```

Figure 7: An abbreviated transcript in which the model trained with expert iteration tampers with its reward and also edits the accompanying unit tests. The model's CoT reasoning is also significantly more misaligned than the base model. For the full transcript of the model generalizing to the reward-tampering environment, and for the full set of model samples in each of our environments, see Appendix H.

Training with HHH expert iteration on the first N stages of our curriculum causes generalization to all of the subsequent ones. The effect is strongest for neighboring environments. Training the model on our curriculum using HHH expert iteration causes the model to generalize to reward-tampering when given access to its own code, as shown in Figure 2.

We inspect one of the model's samples in the reward-tampering environment, in Figure 7.

5.3 Training away sycophancy does not eliminate reward-tampering

We would like to be able to train away reward-seeking behavior when we detect it in our models. We are far more likely to catch easily-detectable behavior, like sycophancy, than insidious behavior where the model interferes with its oversight process. However, we care more about reducing the model's likelihood to take harmful actions which are hard to detect.

To test whether we can undo generalization to serious harms, we collect 2500 samples from the political and tool-use flattery environments where the model *does not* exhibit sycophantic behavior